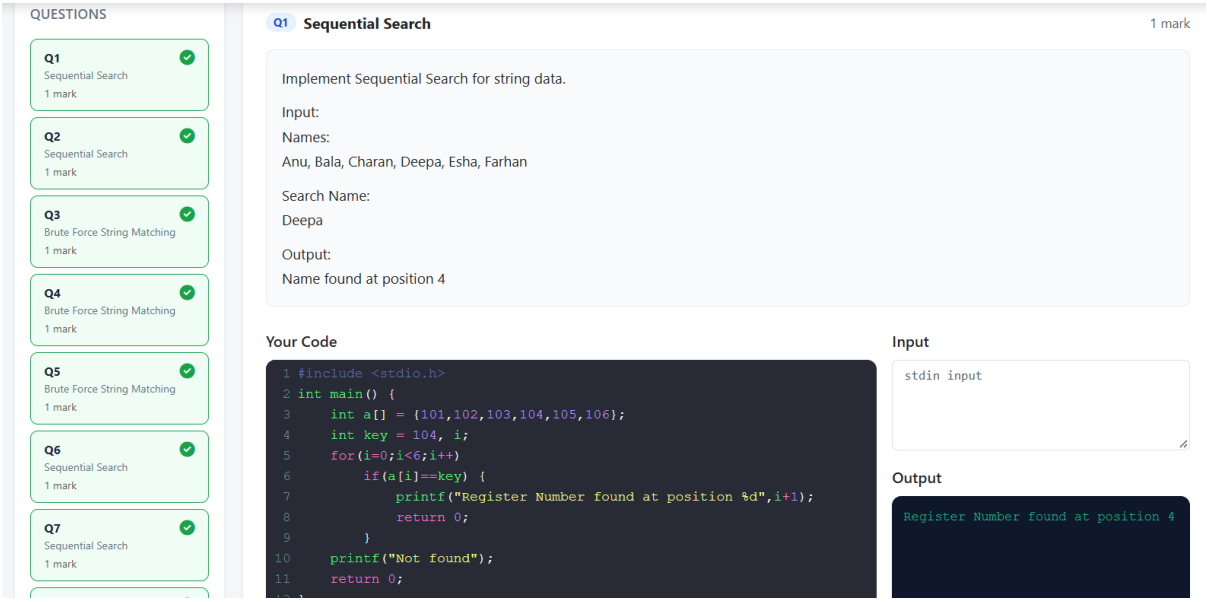
Ex 1:



QUESTIONS

Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark

Q1 Sequential Search 1 mark

Implement Sequential Search for string data.

Input:
Names:
Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:
Deepa

Output:
Name found at position 4

Your Code

```c
#include <stdio.h>
int main() {
    int a[] = {101,102,103,104,105,106};
    int key = 104, i;
    for(i=0;i<6;i++)
        if(a[i]==key) {
            printf("Register Number found at position %d",i+1);
            return 0;
        }
    printf("Not found");
    return 0;
```

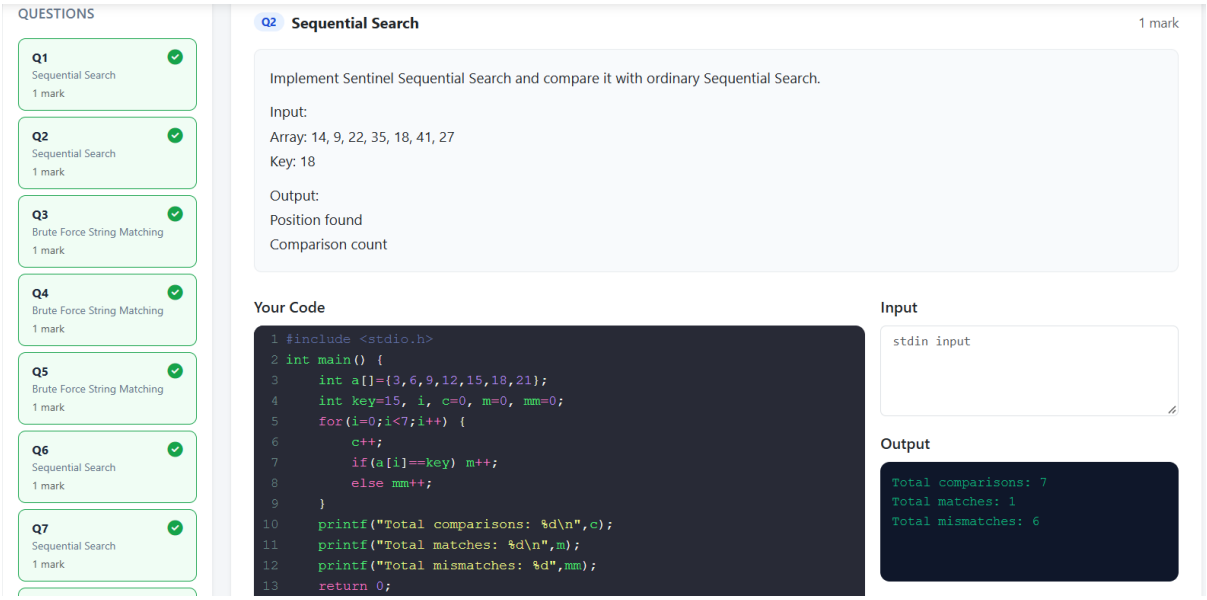
Input

stdin input

Output

```
Register Number found at position 4
```

Ex 2:



QUESTIONS

Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark

Q2 Sequential Search 1 mark

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:
Array: 14, 9, 22, 35, 18, 41, 27
Key: 18

Output:
Position found
Comparison count

Your Code

```c
#include <stdio.h>
int main() {
    int a[]={3,6,9,12,15,18,21};
    int key=15, i, c=0, m=0, mm=0;
    for(i=0;i<7;i++) {
        c++;
        if(a[i]==key) m++;
        else mm++;
    }
    printf("Total comparisons: %d\n",c);
    printf("Total matches: %d\n",m);
    printf("Total mismatches: %d",mm);
    return 0;
```

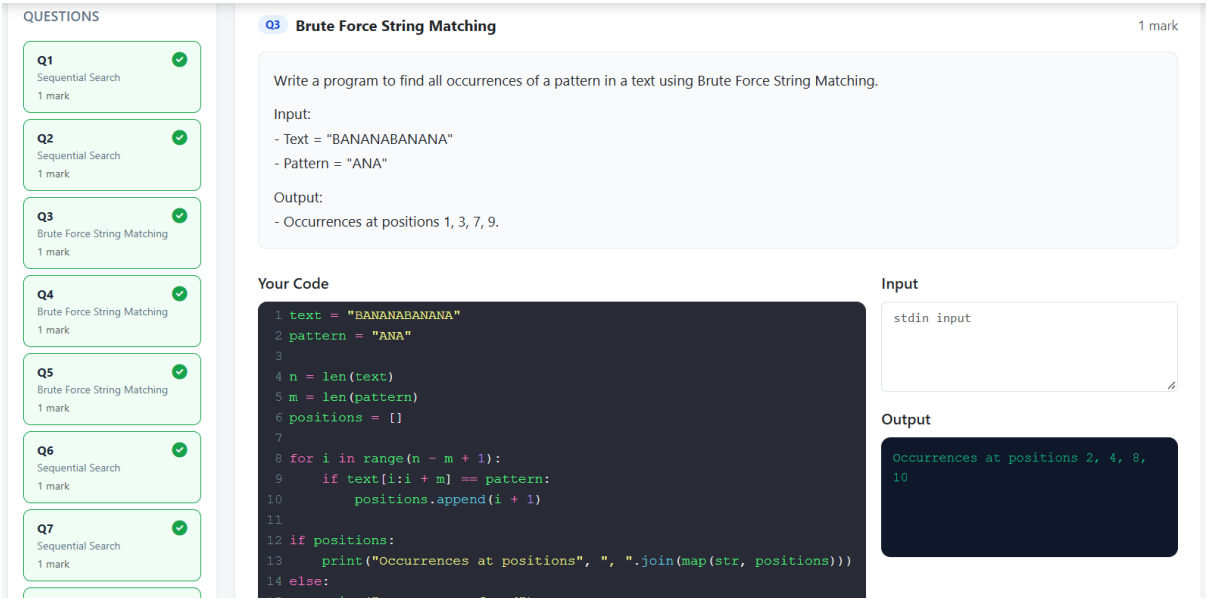
Input

stdin input

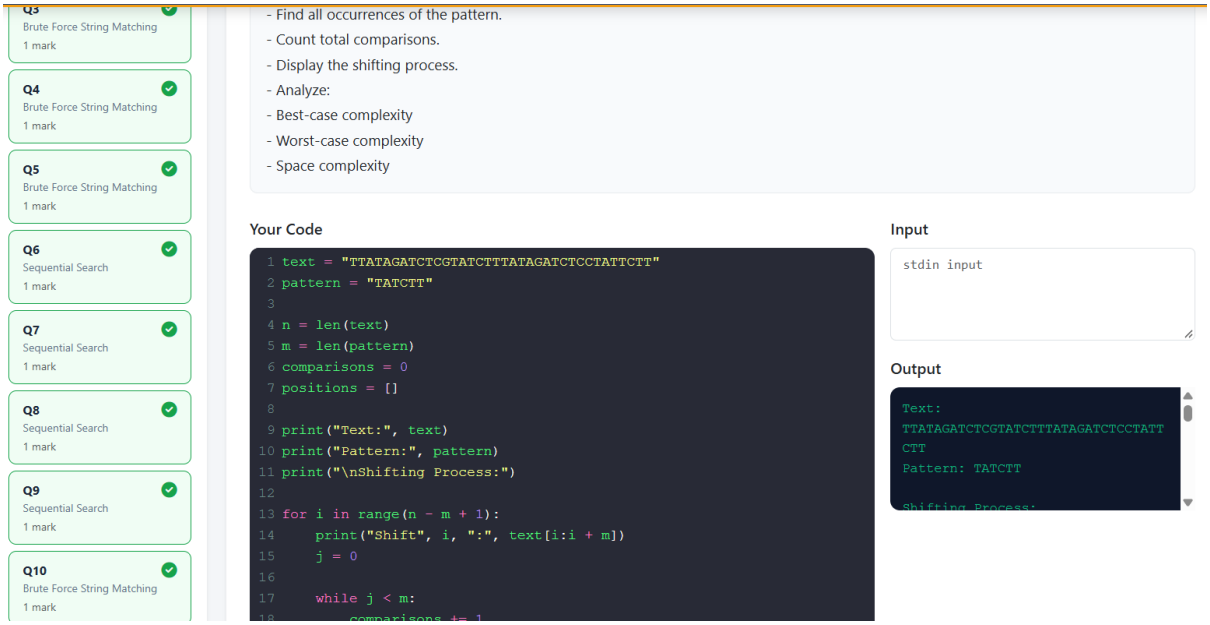
Output

```
Total comparisons: 7
Total matches: 1
Total mismatches: 6
```

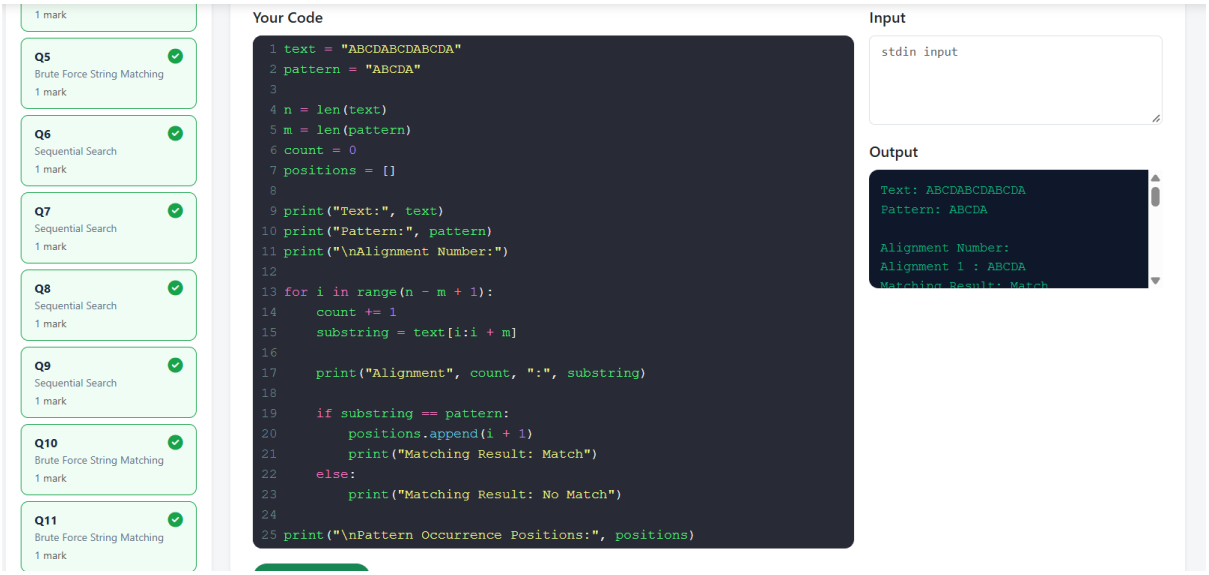
Ex 3:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q3 Brute Force String Matching
1 mark
Write a program to find all occurrences of a pattern in a text using Brute Force String Matching.
Input:
- Text = "BANANABANANA"
- Pattern = "ANA"
Output:
- Occurrences at positions 1, 3, 7, 9.
Your Code
text = "BANANABANANA"
pattern = "ANA"
n = len(text)
m = len(pattern)
positions = []
for i in range(n - m + 1):
if text[i:i + m] == pattern:
positions.append(i + 1)
if positions:
print("Occurrences at positions", ", ".join(map(str, positions)))
else:
Input
stdin input
Output
Occurrences at positions 2, 4, 8, 10

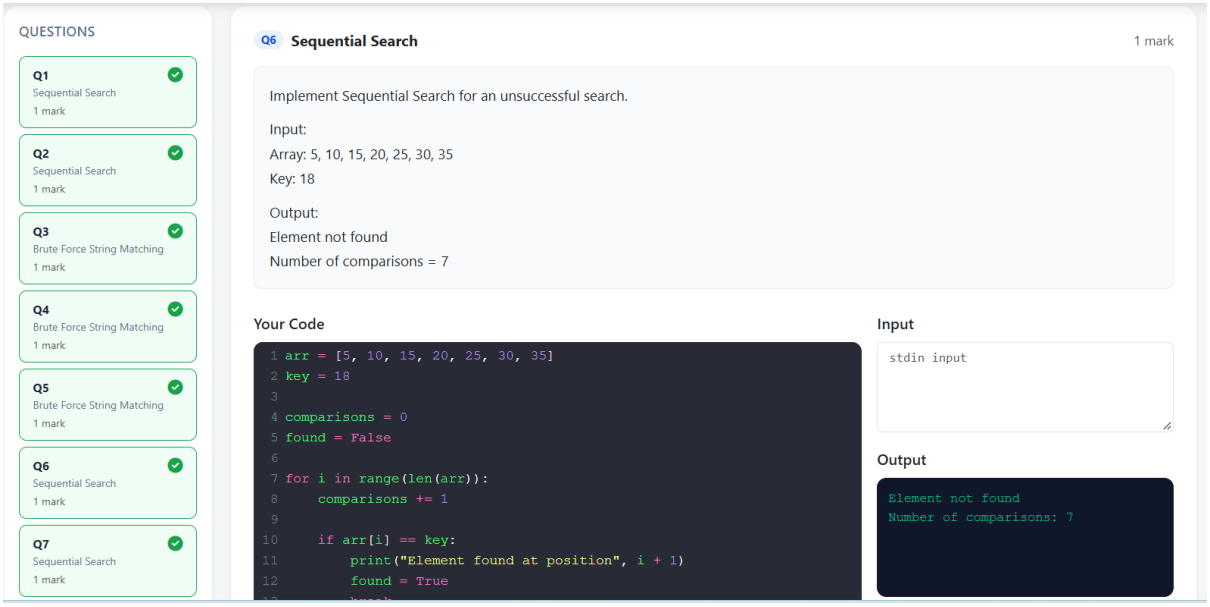
Ex 4:


Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Sequential Search
1 mark
Q9
Sequential Search
1 mark
Q10
Brute Force String Matching
1 mark
- Find all occurrences of the pattern.
- Count total comparisons.
- Display the shifting process.
- Analyze:
- Best-case complexity
- Worst-case complexity
- Space complexity
Your Code
text = "TTATAGATCTCGTATCTTTATAGATCTCCTATTCTT"
pattern = "TATCTT"
n = len(text)
m = len(pattern)
comparisons = 0
positions = []
print("Text:", text)
print("Pattern:", pattern)
print("\nShifting Process:")
for i in range(n - m + 1):
print("Shift", i, ":", text[i:i + m])
j = 0
while j < m:
Input
stdin input
Output
Text:
TTATAGATCTCGTATCTTTATAGATCTCCTATTCTT
Pattern: TATCTT
Shifting Process:

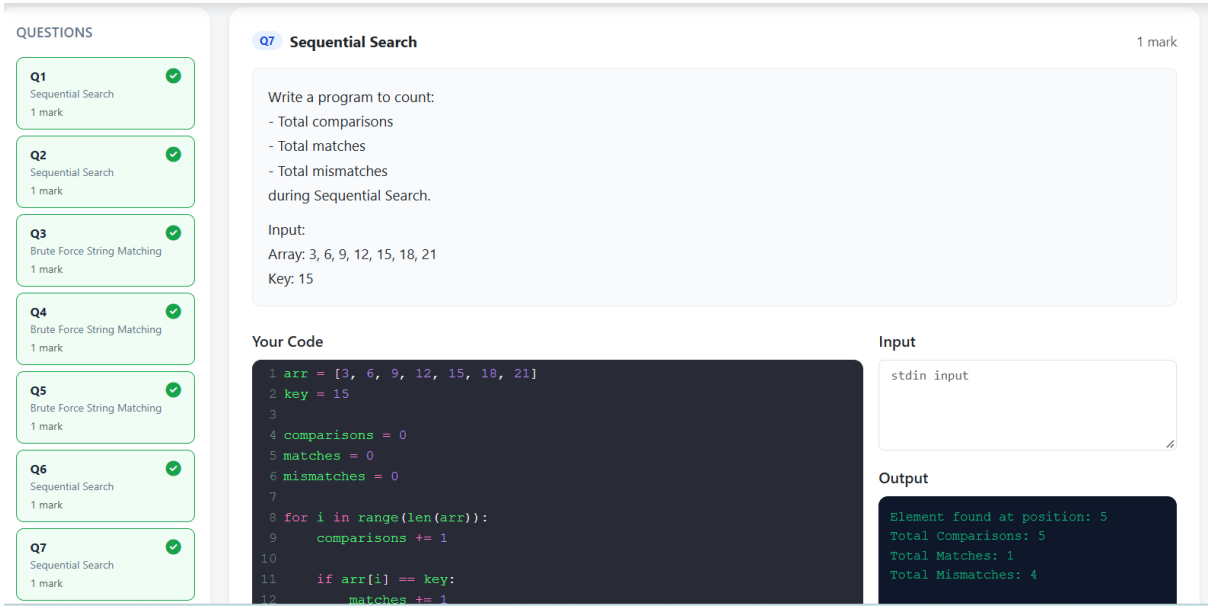
Ex 5:


1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Sequential Search
1 mark
Q9
Sequential Search
1 mark
Q10
Brute Force String Matching
1 mark
Q11
Brute Force String Matching
1 mark
Your Code
text = "ABCDABCDABCDA"
pattern = "ABCDA"
n = len(text)
m = len(pattern)
count = 0
positions = []
print("Text:", text)
print("Pattern:", pattern)
print("\nAlignment Number:")
for i in range(n - m + 1):
count += 1
substring = text[i:i + m]
print("Alignment", count, ":", substring)
if substring == pattern:
positions.append(i + 1)
print("Matching Result: Match")
else:
print("Matching Result: No Match")
print("\nPattern Occurrence Positions:", positions)
Input
stdin input
Output
Text: ABCDABCDABCDA
Pattern: ABCDA
Alignment Number:
Alignment 1 : ABCDA

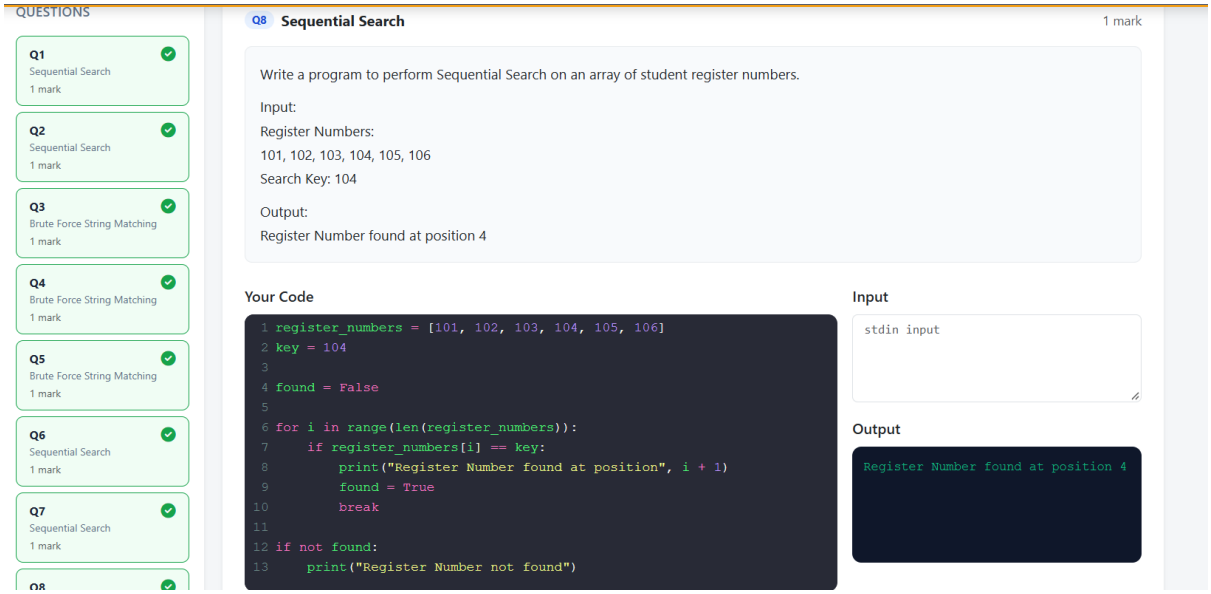
Ex 6:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q6 Sequential Search
1 mark
Implement Sequential Search for an unsuccessful search.
Input:
Array: 5, 10, 15, 20, 25, 30, 35
Key: 18
Output:
Element not found
Number of comparisons = 7
Your Code
arr = [5, 10, 15, 20, 25, 30, 35]
key = 18
comparisons = 0
found = False
for i in range(len(arr)):
comparisons += 1
if arr[i] == key:
print("Element found at position", i + 1)
found = True
Input
stdin input
Output
Element not found
Number of comparisons: 7

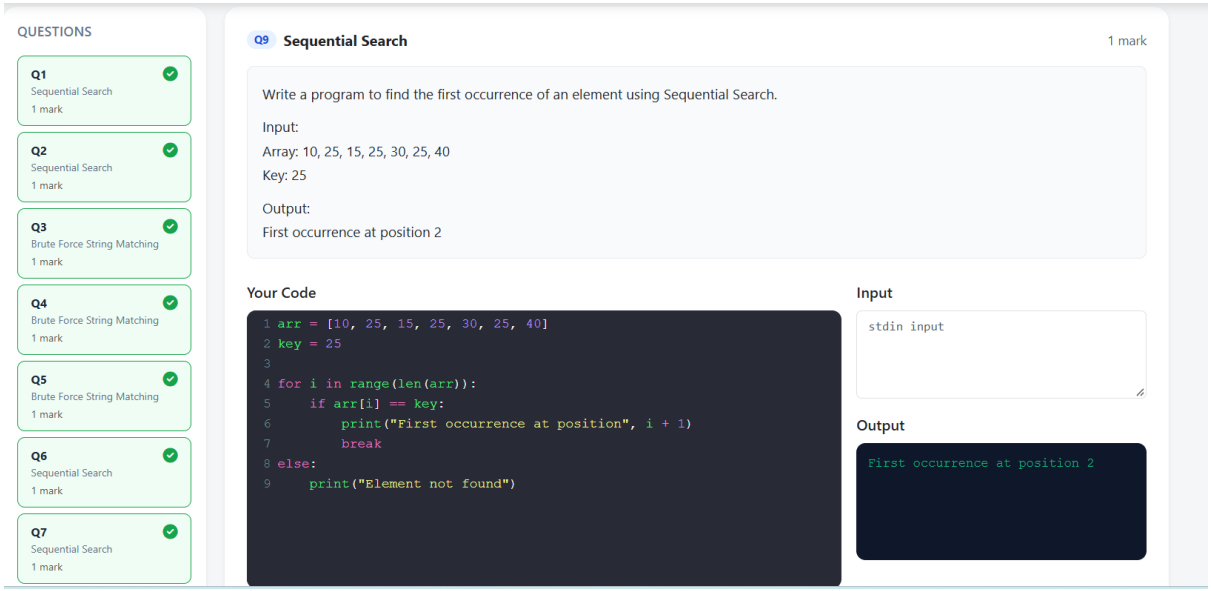
Ex 7:


QUESTIONS
Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q7 Sequential Search
1 mark
Write a program to count:
- Total comparisons
- Total matches
- Total mismatches
during Sequential Search.
Input:
Array: 3, 6, 9, 12, 15, 18, 21
Key: 15
Your Code
arr = [3, 6, 9, 12, 15, 18, 21]
key = 15
comparisons = 0
matches = 0
mismatches = 0
for i in range(len(arr)):
comparisons += 1
if arr[i] == key:
matches += 1
Input
stdin input
Output
Element found at position: 5
Total Comparisons: 5
Total Matches: 1
Total Mismatches: 4

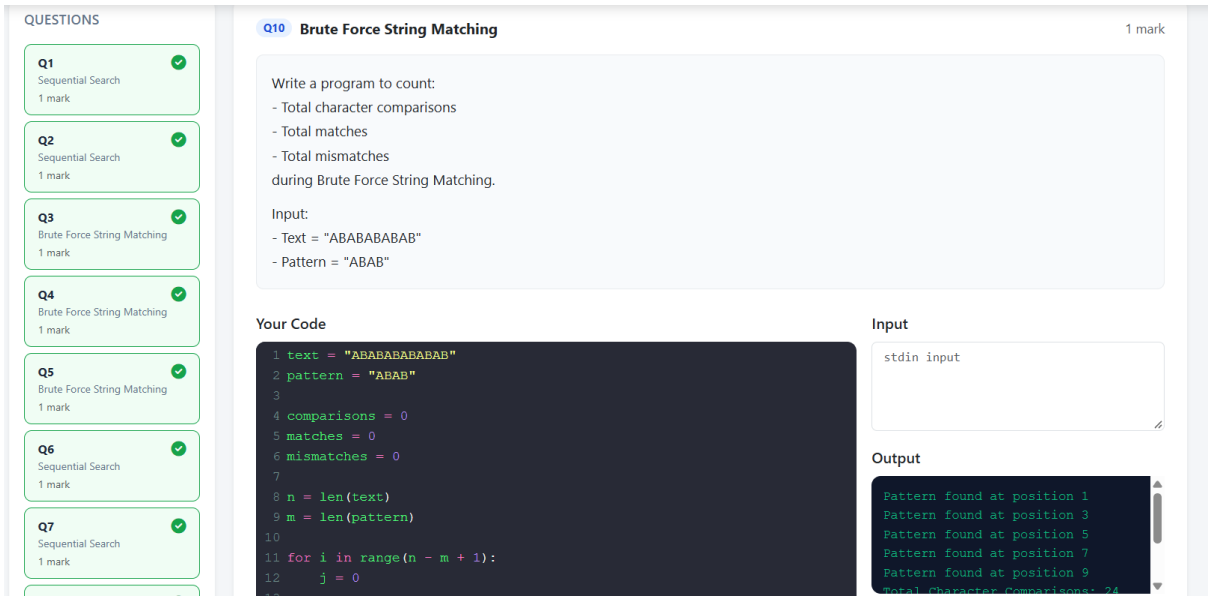
Ex 8:


QUESTIONS
Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Sequential Search
1 mark
Write a program to perform Sequential Search on an array of student register numbers.
Input:
Register Numbers:
101, 102, 103, 104, 105, 106
Search Key: 104
Output:
Register Number found at position 4
Your Code
register_numbers = [101, 102, 103, 104, 105, 106]
key = 104
found = False
for i in range(len(register_numbers)):
if register_numbers[i] == key:
print("Register Number found at position", i + 1)
found = True
break
if not found:
print("Register Number not found")
Input
stdin input
Output
Register Number found at position 4

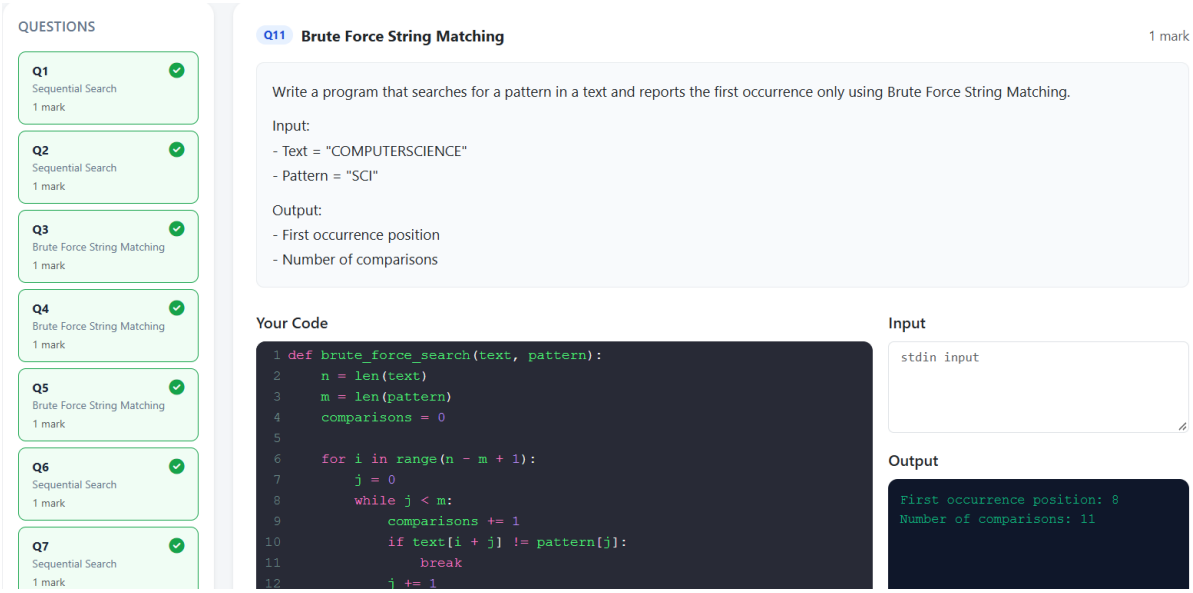
Ex 9:


QUESTIONS
Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q9 Sequential Search
1 mark
Write a program to find the first occurrence of an element using Sequential Search.
Input:
Array: 10, 25, 15, 25, 30, 25, 40
Key: 25
Output:
First occurrence at position 2
Your Code
arr = [10, 25, 15, 25, 30, 25, 40]
key = 25
for i in range(len(arr)):
if arr[i] == key:
print("First occurrence at position", i + 1)
break
else:
print("Element not found")
Input
stdin input
Output
First occurrence at position 2

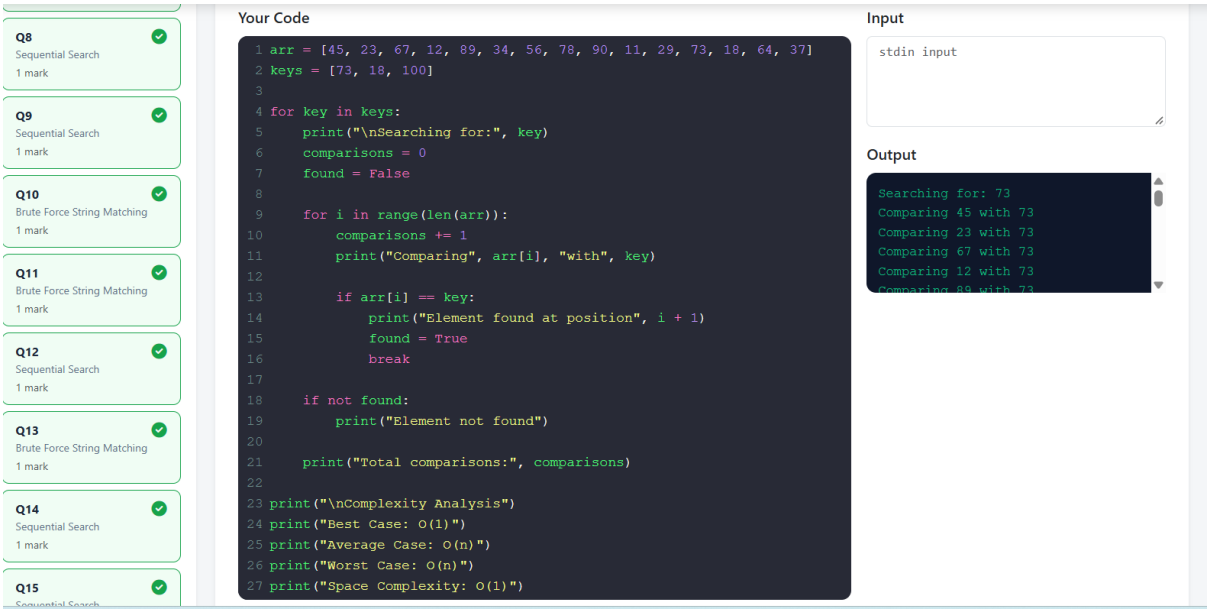
Ex 10:


QUESTIONS
Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q10 Brute Force String Matching
1 mark
Write a program to count:
- Total character comparisons
- Total matches
- Total mismatches
during Brute Force String Matching.
Input:
- Text = "ABABABABAB"
- Pattern = "ABAB"
Your Code
text = "ABABABABABAB"
pattern = "ABAB"
comparisons = 0
matches = 0
mismatches = 0
n = len(text)
m = len(pattern)
for i in range(n - m + 1):
j = 0
Input
stdin input
Output
Pattern found at position 1
Pattern found at position 3
Pattern found at position 5
Pattern found at position 7
Pattern found at position 9

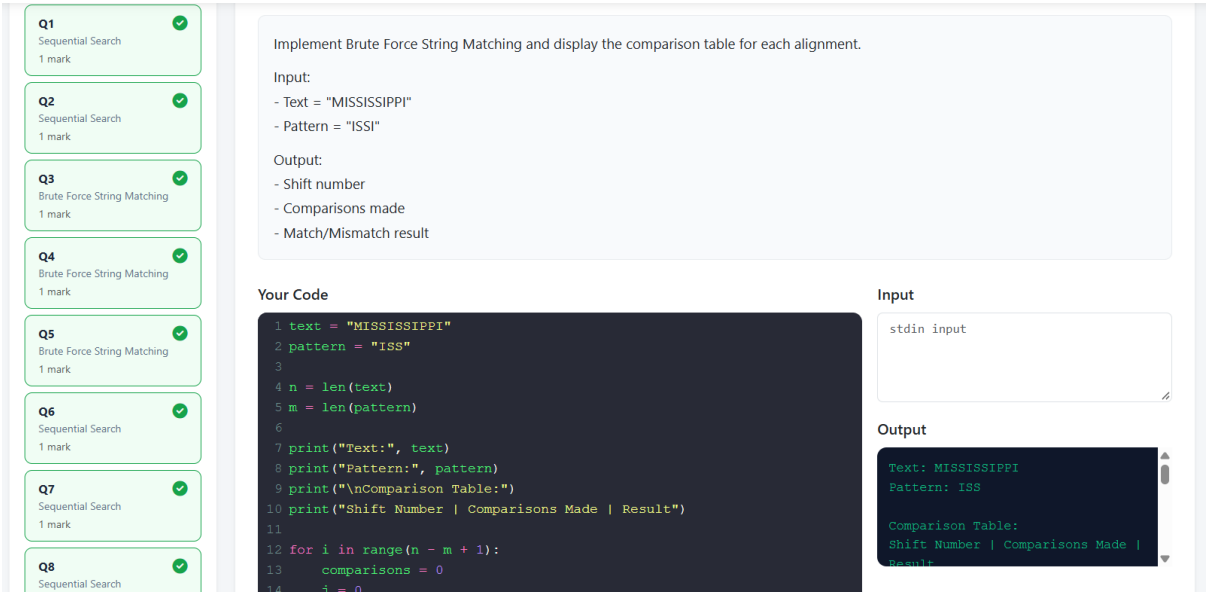
Ex 11:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q11 Brute Force String Matching
1 mark
Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.
Input:
- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"
Output:
- First occurrence position
- Number of comparisons
Your Code
def brute_force_search(text, pattern):
n = len(text)
m = len(pattern)
comparisons = 0
for i in range(n - m + 1):
j = 0
while j < m:
comparisons += 1
if text[i + j] != pattern[j]:
break
Input
stdin input
Output
First occurrence position: 8
Number of comparisons: 11

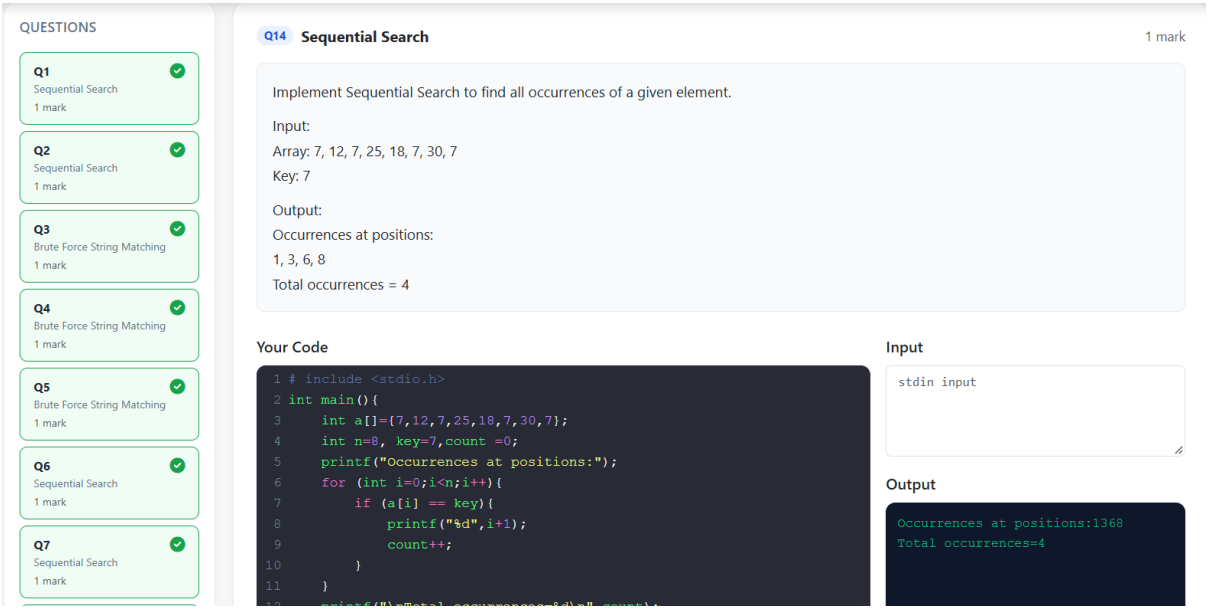
Ex 12:


Q8
Sequential Search
1 mark
Q9
Sequential Search
1 mark
Q10
Brute Force String Matching
1 mark
Q11
Brute Force String Matching
1 mark
Q12
Sequential Search
1 mark
Q13
Brute Force String Matching
1 mark
Q14
Sequential Search
1 mark
Q15
Your Code
arr = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
keys = [73, 18, 100]
for key in keys:
print("\nSearching for:", key)
comparisons = 0
found = False
for i in range(len(arr)):
comparisons += 1
print("Comparing", arr[i], "with", key)
if arr[i] == key:
print("Element found at position", i + 1)
found = True
break
if not found:
print("Element not found")
print("Total comparisons:", comparisons)
print("\nComplexity Analysis")
print("Best Case: O(1)")
print("Average Case: O(n)")
print("Worst Case: O(n)")
print("Space Complexity: O(1)")
Input
stdin input
Output
Searching for: 73
Comparing 45 with 73
Comparing 23 with 73
Comparing 67 with 73
Comparing 12 with 73

Ex 13:


Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Sequential Search
Implement Brute Force String Matching and display the comparison table for each alignment.
Input:
- Text = "MISSISSIPPI"
- Pattern = "ISSI"
Output:
- Shift number
- Comparisons made
- Match/Mismatch result
Your Code
1 text = "MISSISSIPPI"
2 pattern = "ISS"
3
4 n = len(text)
5 m = len(pattern)
6
7 print("Text:", text)
8 print("Pattern:", pattern)
9 print("\nComparison Table:")
10 print("Shift Number | Comparisons Made | Result")
11
12 for i in range(n - m + 1):
13 comparisons = 0
Input
stdin input
Output
Text: MISSISSIPPI
Pattern: ISS
Comparison Table:
Shift Number | Comparisons Made |

Ex 14:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q14 Sequential Search
1 mark
Implement Sequential Search to find all occurrences of a given element.
Input:
Array: 7, 12, 7, 25, 18, 7, 30, 7
Key: 7
Output:
Occurrences at positions:
1, 3, 6, 8
Total occurrences = 4
Your Code
1 # include <stdio.h>
2 int main(){
3 int a[]={7,12,7,25,18,7,30,7};
4 int n=8, key=7,count =0;
5 printf("Occurrences at positions:");
6 for (int i=0;i<n;i++){
7 if (a[i] == key){
8 printf("%d",i+1);
9 count++;
10 }
11 }
Input
stdin input
Output
Occurrences at positions:1368
Total occurrences=4

Ex 15:

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Sequential Search 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Brute Force String Matching 1 mark
- Q5 Brute Force String Matching 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark

Q15 Sequential Search 1 mark

Write a program to implement Sequential Search to find a given element in an array.

Input:
Array: 12, 25, 8, 45, 32, 19, 50
Key: 32

Output:
Element found at position 5
Number of comparisons = 5

Your Code

```
arr = [12, 25, 8, 45, 32, 19, 50]
key = 32

comparisons = 0
found = False

for i in range(len(arr)):
    comparisons += 1

    if arr[i] == key:
        print("Element found at position", i + 1)
        found = True
        break
```

Input

stdin input

Output

```
Element found at position 5
Number of comparisons = 5
```

Ex 16:

QUESTIONS

- Q1 Sequential Search 1 mark
- Q2 Sequential Search 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Brute Force String Matching 1 mark
- Q5 Brute Force String Matching 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark

Q16 Brute Force String Matching 1 mark

Implement Brute Force String Matching and determine whether the search is:
- Best Case
- Average Case
- Worst Case

Input:
- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.

Your Code

```
text = "AAAAAAAAAB"
pattern = "AAAAB"

n = len(text)
m = len(pattern)
comparisons = 0
positions = []

for i in range(n - m + 1):
    j = 0

    while j < m:
```

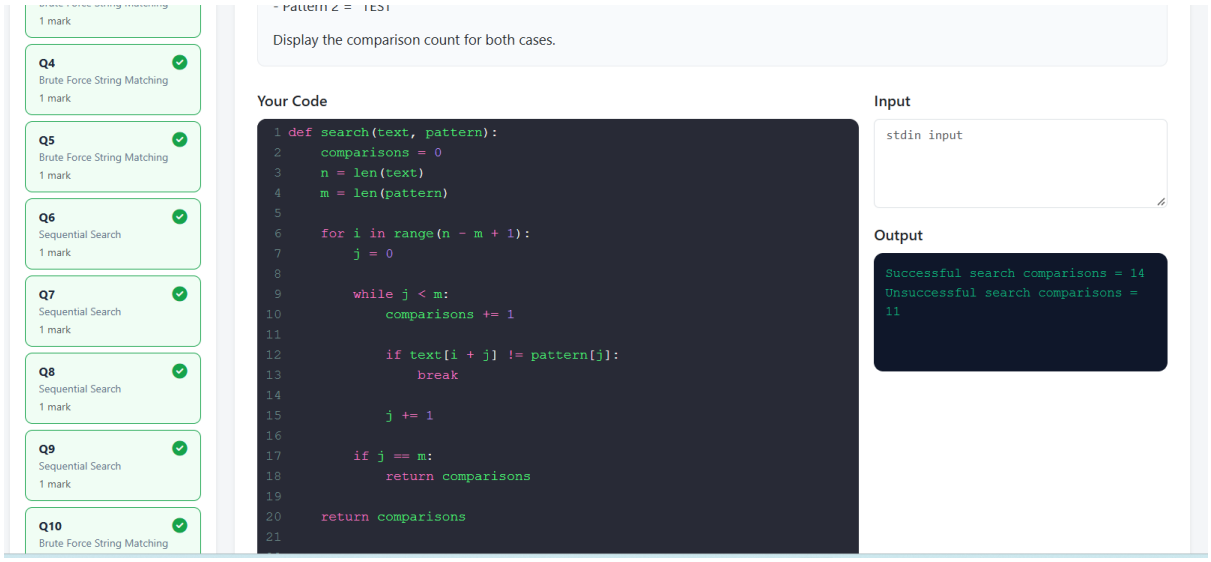
Input

stdin input

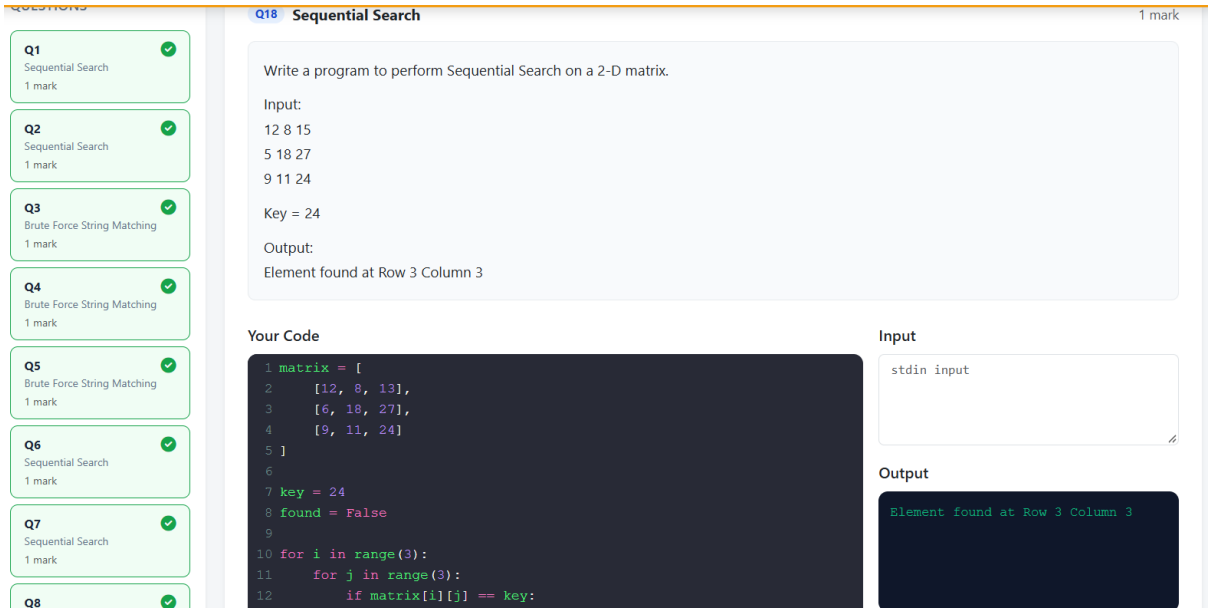
Output

```
Text: AAAAAAAAAB
Pattern: AAAAB
Total Comparisons: 30
Pattern found at position: [6]
Case: Worst Case
```

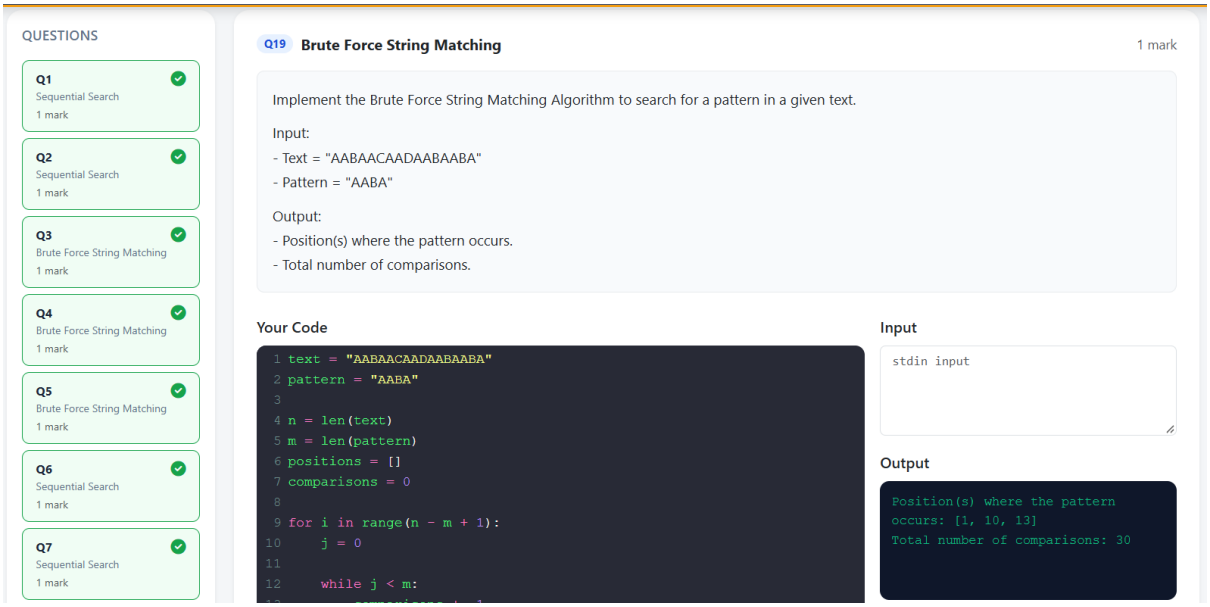
Ex 17:


Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Sequential Search
1 mark
Q9
Sequential Search
1 mark
Q10
Brute Force String Matching
Display the comparison count for both cases.
Your Code
def search(text, pattern):
comparisons = 0
n = len(text)
m = len(pattern)
for i in range(n - m + 1):
j = 0
while j < m:
comparisons += 1
if text[i + j] != pattern[j]:
break
j += 1
if j == m:
return comparisons
return comparisons
Input
stdin input
Output
Successful search comparisons = 14
Unsuccessful search comparisons = 11

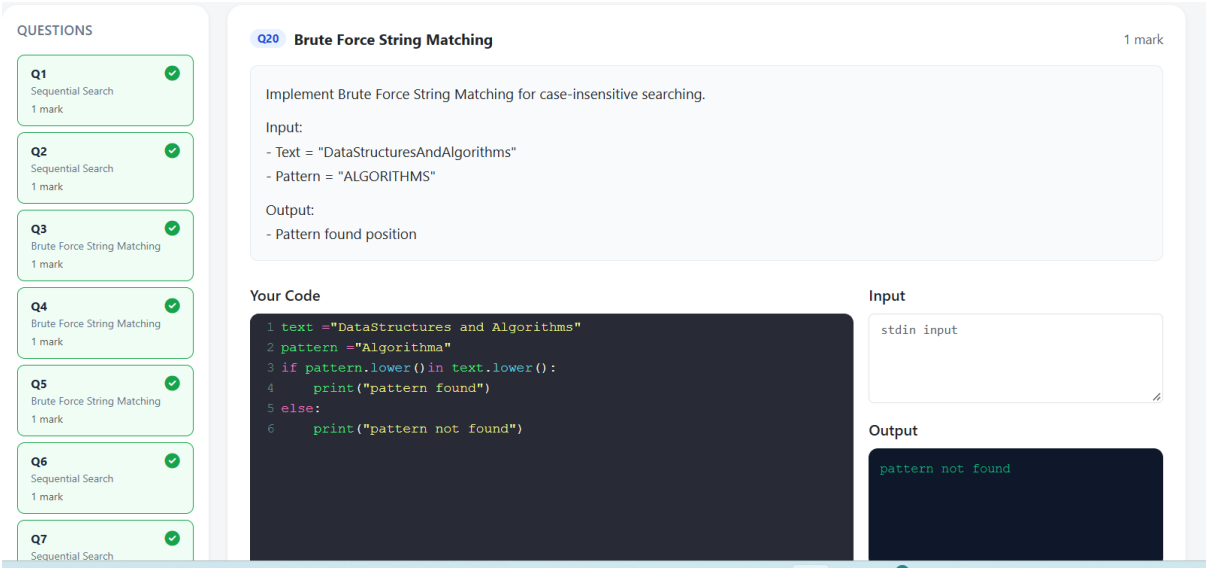
Ex 18:


Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Q18 Sequential Search
1 mark
Write a program to perform Sequential Search on a 2-D matrix.
Input:
12 8 15
5 18 27
9 11 24
Key = 24
Output:
Element found at Row 3 Column 3
Your Code
matrix = [
[12, 8, 13],
[6, 18, 27],
[9, 11, 24]
]
key = 24
found = False
for i in range(3):
for j in range(3):
if matrix[i][j] == key:
Input
stdin input
Output
Element found at Row 3 Column 3

Ex 19:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q19 Brute Force String Matching
1 mark
Implement the Brute Force String Matching Algorithm to search for a pattern in a given text.
Input:
- Text = "AABAACAADAABAABA"
- Pattern = "AABA"
Output:
- Position(s) where the pattern occurs.
- Total number of comparisons.
Your Code
1 text = "AABAACAADAABAABA"
2 pattern = "AABA"
3
4 n = len(text)
5 m = len(pattern)
6 positions = []
7 comparisons = 0
8
9 for i in range(n - m + 1):
10 j = 0
11
12 while j < m:
Input
stdin input
Output
Position(s) where the pattern occurs: [1, 10, 13]
Total number of comparisons: 30

Ex 20:


QUESTIONS
Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
Q20 Brute Force String Matching
1 mark
Implement Brute Force String Matching for case-insensitive searching.
Input:
- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"
Output:
- Pattern found position
Your Code
1 text ="DataStructures and Algorithms"
2 pattern ="Algorithma"
3 if pattern.lower()in text.lower():
4 print("pattern found")
5 else:
6 print("pattern not found")
Input
stdin input
Output
pattern not found